



Tunisian Republic
Ministry of Higher Education and Scientific
Research
University of Sousse
**Higher Institute of Computer Science
and Communication Technologies
(ISITCOM)**



End of Studies Project Report

Submitted in partial fulfillment of the requirements for the
National Engineering Degree — Specialty: Teleinformatics

Presented by:

Asma Daoussi

Development of a Digital Asset Marketplace

for Inherited Games Studio (IGS)

Carried out at: Inherited Games Studio (IGS), Tunisia

Academic Supervisor: Mr. Sami Ben Amor

Company Supervisor: Mrs. Nour Ltaief

Defended on /06/2026 **before the jury composed of:**

Jury President: Mr. _____

Rapporteur: Mr. _____

Academic Year: 2025–2026

Authorization for Defense

Company Supervisor

I hereby authorize the student to submit their End of Studies Project Report for the jury defense.

Mrs Nour LTAIEF

Company Supervisor, Inherited Games Studio

Signature: _____

Academic Supervisor

I hereby authorize the student to submit their End of Studies Project Report for the jury defense.

Mr Sami BEN AMOR

Academic Supervisor, ISITCOM — University of Sousse

Signature: _____

Dedication

*To my beloved family,
who have been my unwavering pillar of strength
throughout every step of this journey.*

*To my parents,
whose sacrifices, endless encouragement, and unconditional love
have been the foundation upon which I built every dream.*

*To my brothers and sisters,
for the laughter, the support, and the belief
that always pushed me forward.*

*To my friends and colleagues,
who made these long study years memorable
and filled them with moments of joy and solidarity.*

*To every person who believed in me
even when I doubted myself —*

this work is for you.

Asma Daoussi

Acknowledgments

First and foremost, I would like to express my sincere gratitude to **Mr. Sami Ben Amor**, my academic supervisor, for his continuous guidance, insightful advice, and unwavering support throughout the realization of this project. His expertise and constructive feedback have been invaluable in shaping both the technical direction and the quality of this work.

I extend my heartfelt thanks to **Mrs Nour Ltaief**, my company supervisor at Inherited Games Studio, for welcoming me into the team, sharing her professional experience, and providing the resources and environment necessary to bring this project to life. Her trust in my capabilities and her constant encouragement made this internship an enriching and rewarding experience.

I would like to express my appreciation to all the staff at **Inherited Games Studio (IGS)** for their warm welcome and their readiness to assist me whenever needed. Working alongside such a talented and passionate team has been a privilege.

My gratitude also goes to the faculty and staff of **ISITCOM — Higher Institute of Computer Science and Communication Technologies** at the **University of Sousse**, whose dedication to quality education provided me with the theoretical and practical foundations that made this project possible.

I am also deeply thankful to the members of the **jury** for dedicating their time to evaluate this work, and for the honor they grant me by participating in its defense.

Finally, I would like to thank my family and friends for their patience, encouragement, and moral support during the long nights and challenging moments of this project.

Asma Daoussi
Sousse, 2026

Contents

Dedication	i
Acknowledgments	ii
List of Figures	vi
List of Tables	vii
List of Acronyms	viii
General Introduction	1
1 General Context	3
1.1 Host Organization: Inherited Games Studio	4
1.2 Problem Statement	4
1.3 Comparative Study of Existing Solutions	5
1.4 Proposed Solution	5
1.5 Development Methodology: Scrum	6
1.6 Technology Stack	7
1.7 Work Environment	7
2 Requirements Analysis	9
2.1 Actors Identification	10
2.2 Needs Identification	11
2.2.1 Functional Requirements	11
2.3 Non-Functional Requirements	13
2.4 Project Management with Scrum	13
2.4.1 Product Backlog	13
2.4.2 Sprint Planning	16
2.5 Logical Architecture	16
2.5.1 Software Architecture	17
2.6 Design Patterns	17
2.7 Global Use Case Diagram	19
2.8 Global Class Diagram	19
2.9 Key Sequence Diagrams	19

3	Sprint 0: Research, Planning & Architecture	21
3.1	System Architecture	22
3.2	Technology Stack Justification	23
3.3	Database Design	23
4	Sprints 1–2: Core Platform	25
4.1	Sprint 1: Authentication, Users and RBAC	26
4.1.1	Sprint Backlog	26
4.1.2	Analysis and Design	26
4.1.3	Implementation	27
4.1.4	Sprint Review	27
4.2	Sprint 2: Catalog and Marketplace Foundation	27
4.2.1	Sprint Backlog	27
4.2.2	Analysis and Design	28
4.2.3	Implementation	28
4.2.4	Sprint Review	28
5	Sprints 3–4: Marketplace Experience	29
5.1	Sprint 3: Upload, Moderation and Trust	30
5.1.1	Sprint Backlog	30
5.1.2	Analysis and Design	30
5.1.3	Implementation	31
5.1.4	Sprint Review	31
5.2	Sprint 4: Intern Workflow and Community Modules	31
5.2.1	Sprint Backlog	31
5.2.2	Analysis and Design	32
5.2.3	Implementation	32
5.2.4	Sprint Review	32
6	Sprints 5–6: Workspace and AI	33
6.1	Sprint 5: IGS Workspace	34
6.1.1	Sprint Backlog	34
6.1.2	Analysis and Design	34
6.1.3	Implementation	35
6.1.4	Sprint Review	35
6.2	Sprint 6: AI Services	35
6.2.1	Sprint Backlog	35
6.2.2	Analysis and Design	35
6.2.3	Implementation	36
6.2.4	Sprint Review	36

7	Sprints 7–8: IGS Workspace & Admin Dashboard	37
7.1	Sprint 8: Admin Dashboard & Premium UI Overhaul	38
7.1.1	Sprint Backlog	38
7.1.2	Implementation	39
7.1.3	Sprint Review — Deliverables	39
8	CI/CD & Deployment	40
8.1	CI/CD Pipeline	41
8.1.1	Deployment Pipeline	41
8.2	Containerization with Docker	41
8.3	Nginx Reverse Proxy	42
8.4	Deployment Strategy	42
8.4.1	Image Publication and Remote Update	42
8.4.2	Database Migrations	42
8.4.3	Environment Secrets	43
8.5	API Documentation	43
8.6	Testing Strategy	44
	General Conclusion	45
	Bibliography	46
	Webography	47
	Abstract	48

List of Figures

2.1	Three-Tier Logical Architecture of IGS Marketplace	17
2.2	Global Use Case Diagram	19
8.1	IGS Marketplace Deployment Pipeline	41

List of Tables

1.1	Inherited Games Studio — Company Profile	4
1.2	Comparative Analysis of Existing 3D Asset Marketplace Platforms . .	5
1.3	Sprint Planning Overview	6
1.4	Technology Stack Summary	7
1.5	Software Work Environment	8
2.1	System Actors and Permission Levels	10
2.2	Non-Functional Requirements	13
2.3	Product Backlog of IGS Marketplace	13
2.4	Sprint Planning Overview	16
2.5	Design Patterns Used in the Project	18
3.1	Sprint 0 — Technology Stack	23
4.1	Sprint 1 Backlog	26
4.2	Sprint 2 Backlog	27
5.1	Sprint 3 Backlog	30
5.2	Sprint 4 Backlog	31
6.1	Sprint 5 Backlog	34
6.2	Sprint 6 Backlog	35
7.1	Sprint 8 — User Stories	38
8.1	GitHub Actions Secrets — Classification	43
8.2	API Documentation — Collections by Domain	43
8.3	Test Coverage by Layer	44

List of Acronyms

Acronym	Definition
API	Application Programming Interface
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DB	Database
DM	Direct Message
ERP	Enterprise Resource Planning
GLB	GL Binary (3D format)
GLTF	GL Transmission Format
HDRI	High Dynamic Range Imaging
HTTP	HyperText Transfer Protocol
IGS	Inherited Games Studio
ISITCOM	Institut Supérieur de l'Informatique et des Technologies de Communication
JWT	JSON Web Token
MVC	Model-View-Controller
OAuth	Open Authorization
ORM	Object-Relational Mapping
PBR	Physically Based Rendering
PFE	Projet de Fin d'Études (End of Studies Project)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
SSO	Single Sign-On
UML	Unified Modeling Language
UUID	Universally Unique Identifier

General Introduction

The global video game industry has experienced remarkable growth over the past decade, evolving from a niche entertainment segment into one of the most lucrative creative economies in the world. With revenues surpassing hundreds of billions of dollars annually and a continuously expanding ecosystem of developers, studios, and independent creators, the demand for high-quality digital assets has never been greater. Beyond the games themselves, a significant and often overlooked opportunity lies not in the creation of video games directly, but in building the infrastructure that supports them, namely, digital marketplaces dedicated to the exchange of 3D assets. These platforms serve as essential bridges between creators and consumers, enabling studios, developers, architects, and designers to access a vast library of ready-made models, textures, animations, and environments. The rise of real-time 3D technologies, game engines such as Unity and Unreal Engine, and the expanding metaverse economy has further accelerated the need for structured, accessible, and reliable 3D asset marketplace solutions. Investing in such platforms represents a strategic and high-value proposition in today's digital economy, as they reduce production costs, shorten development cycles, and democratize access to professional-grade visual content.

For this purpose, this project introduces a unified 3D digital marketplace — a centralized platform, or *vitrine*, designed to connect creators and consumers of 3D assets within a single, cohesive ecosystem. The platform caters to both internal users, such as affiliated studios, developers, and in-house teams, and external users, including independent designers, clients, and businesses seeking ready-to-use digital content. Through this marketplace, users can seamlessly browse, upload, purchase, and sell 3D assets across a variety of categories, with a streamlined experience that prioritizes accessibility, quality, and trust. By unifying these interactions under one platform, the solution aims to address the fragmentation currently characterizing the 3D asset market, offering a reliable, scalable, and user-centered alternative to existing tools.

This report documents the different steps of the project towards its realization. It starts with **Chapter 1**, which establishes the general project context, the host organization, a comparative study of existing solutions, the chosen methodology, and the technology stack. **Chapter 2** details the requirements analysis, system actors, functional and non-functional requirements, and global system architecture. **Chapter 3** covers Sprint 0 — the research, planning, and architectural decisions made before any code was written. **Chapters 4 and 5** document Sprints 1 through 4, delivering the core

platform: authentication, RBAC, asset catalog, payment, reviews, and administration. **Chapters 6 and 7** cover Sprints 5 through 8, delivering the IGS Workspace, AI integration, the recommendation engine, and the final UI overhaul. **Chapter 8** presents the CI/CD pipeline, Docker containerization, and production deployment strategy.

General Context

Sommaire

1.1	Host Organization: Inherited Games Studio	4
1.2	Problem Statement	4
1.3	Comparative Study of Existing Solutions	5
1.4	Proposed Solution	5
1.5	Development Methodology: Scrum	6
1.6	Technology Stack	7
1.7	Work Environment	7

Introduction

This chapter presents the host organization, the problem statement, a comparative study of existing solutions, the proposed solution, the chosen development methodology, and the technology stack.

1.1 Host Organization: Inherited Games Studio

Inherited Games Studio (IGS) is a Tunisian independent video game development company specializing in narrative-driven and multiplayer games. The studio brings together artists, developers, designers, and sound engineers focused on immersive interactive experiences.

Table 1.1: Inherited Games Studio — Company Profile

Field	Information
Industry	Video Game Development / Digital Content
Location	Tunisia
Specialization	3D game development, digital assets, interactive media
Company Supervisor	Mme. Nour Ltaief

1.2 Problem Statement

Before this project, IGS faced several critical operational pain points:

- **No centralized asset repository:** Assets were scattered across local drives with no unified catalog or search.
- **No monetization platform:** IGS had no mechanism to sell surplus assets externally.
- **No role-based access control:** All internal users shared similar access levels, creating security concerns.
- **Fragmented workflows:** Collaboration relied on disconnected tools with no unified project management.
- **Dependency on foreign platforms:** International platforms (Unity Asset Store, Sketchfab, Fab) imposed dollar-denominated fees and no customization.

1.3 Comparative Study of Existing Solutions

The following table compares the main international 3D asset platforms against IGS Marketplace across the features critical to the studio's needs:

Table 1.2: Comparative Analysis of Existing 3D Asset Marketplace Platforms

Feature	Unity AS	Fab	Sketchfab	CGTrader	Gumroad	IGS MP
3D In-Browser Viewer	Partial	Yes	Yes	No	No	Yes
Multi-Engine Support	No	Yes	Yes	Yes	Yes	Yes
Custom RBAC	No	No	No	No	No	Yes
Internal Team Tools	No	No	No	No	No	Yes
AI Features	No	No	No	No	No	Yes
Self-Hosted / Custom	No	No	No	No	No	Yes
Intern Workflow	No	No	No	No	No	Yes
Real-Time Chat	No	No	No	No	No	Yes
Stripe Payment	No	No	No	No	Yes	Yes
Tunisia- Accessible	Partial	Partial	Partial	Partial	Yes	Yes

No existing platform combines a 3D digital asset marketplace with internal studio management tools, customizable role hierarchies, and AI-powered features in a single self-hosted solution. IGS Marketplace is therefore not merely a custom tool but the **first 3D digital asset marketplace platform designed and developed in Tunisia**.

1.4 Proposed Solution

The proposed solution is a full-stack web platform composed of two interconnected applications:

1. **IGS Marketplace (port 5173):** The public-facing marketplace where sellers list assets, buyers purchase and download content, and administrators moderate

the catalog. It includes a 3D interactive viewer, Stripe checkout, a subscription system, and an AI chatbot assistant.

2. **IGS Workspace (port 5174):** A private internal dashboard exclusively for IGS employees and interns, featuring team management, task tracking, project ERP modules, and a real-time direct messaging system.

Both applications share the same backend API (Express.js on port 3001), the same PostgreSQL database, and the same authentication system (Supabase Auth + JWT), enabling seamless Single Sign-On between the two apps.

1.5 Development Methodology: Scrum

Scrum was selected as the development methodology because requirements emerged progressively throughout the internship, the company supervisor (Mme. Nour Ltaief) naturally assumes the Product Owner role, the academic supervisor (Mr. Sami Ben Amor) assumes the Scrum Master role, and sprint reviews align with mandatory internship progress meetings.

The project was divided into **eight sprints**, each lasting approximately two to three weeks:

Table 1.3: Sprint Planning Overview

Sprint	Theme	Key Deliverables
0	Research & Architecture	Architecture design, DB schema, environment setup
1	Core Infrastructure	Backend, PostgreSQL, JWT auth, RBAC (6 roles)
2	Asset Marketplace Core	Catalog, 3D viewer, seller upload, cart, Stripe
3	Content & Administration	Reviews, notifications, admin panel, subscriptions
4	Advanced Role Features	Intern workflow, seller analytics, trading, groups
5	IGS Workspace	Companion app, task management, real-time chat
6	AI Integration	ML models, sentiment analysis, chatbot
7 & 8	AI Expansion & Polish	Recommendation engine, UI overhaul, admin dashboard

1.6 Technology Stack

Table 1.4: Technology Stack Summary

Layer	Technology	Role / Justification
Frontend	React 18 + Vite	SPA framework — fast HMR, component model
	Three.js / Model Viewer	3D asset viewer in the browser
	TailwindCSS	Utility-first responsive styling
Backend	Node.js + Express.js	REST API — non-blocking, JSON-native
	Socket.io	WebSocket real-time events (chat, notifications)
	HuggingFace / Mistral-7B	AI review analysis, chatbot, recommendations
Data & Auth	PostgreSQL 16	Primary relational database
	Supabase	Scalable file hosting for 3D assets
	Supabase Auth	Authentication, sessions, and JWT-based access
Payments	Stripe	Secure card payment processing
Dev Tools	Git + GitHub	Version control and CI/CD
	Postman + draw.io	API testing and UML diagram preparation

1.7 Work Environment

The project was developed in a modern full-stack environment combining frontend, backend, database, storage, payment, AI, version-control, and documentation tools. The following overview summarizes the main tools used during the internship.

Table 1.5: Software Work Environment

Category	Tools			
Frontend	 React	 Vite	 Three.js	 TailwindCSS
Backend	 Node.js	 Express.js	 Socket.io	 JWT
Data, Auth, Storage	 PostgreSQL	 Supabase	 Supabase Auth	 Stripe
AI and Testing	 HuggingFace	 Postman	 ESLint	 npm
DevOps and Documentation	 GitHub	 Actions	 Docker	 draw.io
Development IDE	 Git	 XeLaTeX	 API Tests	 UML

Conclusion

This chapter established the project context: the host organization IGS and its operational challenges, the comparative analysis confirming the absence of a dedicated 3D marketplace in Tunisia, the proposed two-application solution, the Scrum methodology applied across eight sprints, the technology stack, and the work environment. The following chapter details the functional and non-functional requirements of the system.

Requirements Analysis

Sommaire

2.1	Actors Identification	10
2.2	Needs Identification	11
2.2.1	Functional Requirements	11
2.3	Non-Functional Requirements	13
2.4	Project Management with Scrum	13
2.4.1	Product Backlog	13
2.4.2	Sprint Planning	16
2.5	Logical Architecture	16
2.5.1	Software Architecture	17
2.6	Design Patterns	17
2.7	Global Use Case Diagram	19
2.8	Global Class Diagram	19
2.9	Key Sequence Diagrams	19

Introduction

This chapter defines the functional scope of the IGS Marketplace project. It identifies the actors, describes functional and non-functional requirements, presents the product backlog used to drive the Scrum iterations, and summarizes the main UML design views required before implementation.

2.1 Actors Identification

The platform is used by external marketplace users and by internal IGS collaborators. Access rights are controlled through a role-based access control model (RBAC), where each role receives a set of permissions adapted to its responsibilities.

Table 2.1: System Actors and Permission Levels

Role	Type	Level	Main Responsibilities
Visitor	External	–	Browse public catalog pages, search assets, and view asset details.
member	External	10	Create an account, manage cart, purchase or claim assets, download library items, rate and review owned assets.
seller	External	20	Request seller access, upload assets, manage own listings, follow sales and asset statistics.
intern	Internal	40	Access the internal workspace, request paid assets for supervised use, redeem vouchers, and upload supervised assets.
employee	Internal	60	Moderate content, consult analytics, collaborate in workspace projects and chat channels.
admin	Internal	80	Manage users, assets, orders, categories, moderation queues, role requests, vouchers, and audit information.
super_admin	Internal	100	Full platform control, including system configuration, commission settings, and role governance.

2.2 Needs Identification

Following the presentation style used in the reference report, the needs are captured from the point of view of the system actors. They are separated into functional needs, which describe what the system must do, and non-functional needs, which describe quality and technical constraints.

2.2.1 Functional Requirements

Visitor needs

- Browse the public catalog without authentication.
- Search and filter assets by category, price, rating, file type, and availability.
- Consult asset details, previews, descriptions, formats, tags, seller information, and reviews.
- Create an account using email/password or OAuth authentication.

Member needs

- Authenticate securely and manage personal profile information.
- Add assets to the cart, remove them, and proceed to checkout.
- Purchase paid assets through a payment provider and claim free assets.
- Access purchased or unlocked assets from a personal library.
- Rate, review, like, and download assets when access rights allow it.
- Subscribe to a plan and consult subscription history.
- Join groups, propose trades, and receive in-app notifications.

Seller needs

- Request seller access and wait for administrative validation.
- Upload assets with files, thumbnails, formats, metadata, price, and tags.
- Follow moderation status and correct rejected submissions.
- Request edits on already approved assets.
- Consult personal asset statistics and seller analytics.

Intern and internal collaborator needs

- Access the IGS Workspace according to internal role permissions.
- Request access to paid marketplace assets with a justification.
- Redeem vouchers generated by authorized reviewers.
- Collaborate through internal projects, tasks, documents, dashboards, and real-time chat.

Administrator needs

- Manage users, roles, permissions, role requests, and account status.
- Moderate submitted assets, edit requests, intern uploads, and intern asset requests.
- Manage vouchers, subscriptions, categories, commission configuration, and platform settings.
- Consult platform analytics, audit logs, orders, notifications, and moderation history.

AI-assisted needs

- Analyze review sentiment and help moderators identify suspicious or useful feedback.
- Generate asset recommendations from user events such as views, likes, downloads, and purchases.
- Provide chatbot assistance for marketplace questions and navigation.

2.3 Non-Functional Requirements

Table 2.2: Non-Functional Requirements

Category	Requirements
Security	JWT authentication, hashed passwords, OAuth support, RBAC middleware, protected internal routes, rate limiting, Helmet, CORS rules, audit logging for sensitive actions.
Performance	Indexed database queries, paginated listings, cloud storage for heavy files, client-side loading states, and optimized asset previews.
Reliability	Centralized error handling, transactional order creation, persistent notifications, and controlled moderation state transitions.
Scalability	Stateless REST API, separated React applications, PostgreSQL persistence, Supabase storage, and modular route/controller/repository layers.
Usability	Responsive marketplace screens, clear protected-route feedback, accessible forms, preview components for 3D assets, textures, HDRI, shaders, and media.
Maintainability	Layered backend architecture, reusable React components, isolated services, repository-based SQL access, and explicit database migrations/schema files.

2.4 Project Management with Scrum

Following the structure of the reference reports, the project scope is organized as a product backlog. Each backlog item represents a user story or technical capability, prioritized according to its importance for a usable marketplace and then assigned to a sprint.

2.4.1 Product Backlog

Table 2.3: Product Backlog of IGS Marketplace

ID	Theme	User Story	Priority
PB-01	Authentication	As a user, I can register, login, logout, and access protected routes securely.	High

ID	Theme	User Story	Priority
PB-02	RBAC	As an administrator, I can assign roles and permissions so each user sees only authorized features.	High
PB-03	User Profile	As a user, I can consult and update my personal profile information.	High
PB-04	Catalog	As a visitor, I can browse categories and consult asset details.	High
PB-05	Search and Filters	As a visitor, I can search and filter assets by category, price, rating, file type, and availability.	High
PB-06	Asset Preview	As a buyer, I can preview 3D models, textures, HDRIs, shaders, images, and media before purchase.	High
PB-07	Seller Request	As a member, I can request seller access and provide a justification for review.	Medium
PB-08	Asset Upload	As a seller, I can upload files, thumbnails, metadata, tags, formats, and pricing.	High
PB-09	Moderation	As an admin or employee, I can approve, reject, hide, or request changes on submitted assets.	High
PB-10	Asset Edit Request	As a seller, I can request modifications on approved assets without bypassing moderation.	Medium
PB-11	Cart	As a member, I can add, remove, and clear cart items.	High
PB-12	Payment and Orders	As a buyer, I can pay for selected assets and receive an order record.	High
PB-13	Library and Downloads	As a buyer or authorized user, I can access and download unlocked assets from my library.	High
PB-14	Reviews and Ratings	As a buyer, I can rate and review owned assets.	Medium
PB-15	Notifications	As a user, I can receive and manage notifications about requests, orders, and moderation events.	Medium
PB-16	AI Review Analysis	As an administrator, I can use AI sentiment results to support moderation decisions.	Medium
PB-17	Recommendations	As a user, I can receive personalized asset recommendations.	Medium
PB-18	Chatbot	As a visitor or user, I can ask marketplace questions through an AI chatbot.	Medium
PB-19	Intern Requests	As an intern, I can request access to paid assets for training or production needs.	High
PB-20	Vouchers	As an admin, I can generate vouchers; as a user, I can redeem valid vouchers.	Medium
PB-21	Groups	As a user, I can create, join, and manage community groups.	Low
PB-22	Trades	As a user, I can propose and respond to peer-to-peer asset trades.	Low

ID	Theme	User Story	Priority
PB-23	Subscriptions	As a user, I can view plans, subscribe, cancel, and consult subscription history.	Medium
PB-24	Administration	As an administrator, I can manage users, role requests, assets, categories, vouchers, orders, and settings.	High
PB-25	Analytics and Audit	As an administrator, I can consult analytics and trace sensitive actions through audit logs.	Medium
PB-26	Workspace Projects	As an internal user, I can create projects, assign members, attach documents, and monitor progress.	High
PB-27	Workspace Tasks	As an internal user, I can create, assign, update, and track tasks.	High
PB-28	Workspace Chat	As an internal user, I can communicate through channels and direct messages in real time.	Medium
PB-29	CI/CD	As a developer, I can build, test, and deploy the platform through an automated pipeline.	Medium
PB-30	Documentation	As a maintainer, I can understand the project, API organization, schema, and deployment process.	Medium

2.4.2 Sprint Planning

Table 2.4: Sprint Planning Overview

Sprint	Main Objective	Backlog Items
1	Authentication and access control	PB-01, PB-02, PB-03
2	Catalog discovery and public marketplace foundation	PB-04, PB-05, PB-06, PB-07
3	Seller upload and moderation workflows	PB-08, PB-09, PB-10
4	Commerce, library, reviews, and ratings	PB-11, PB-12, PB-13, PB-14
5	AI services and notifications	PB-15, PB-16, PB-17, PB-18
6	Intern workflow, vouchers, and community modules	PB-19, PB-20, PB-21, PB-22
7	Subscriptions, analytics, and administration	PB-23, PB-24, PB-25
8	IGS Workspace, collaboration, and deployment	PB-26, PB-27, PB-28, PB-29, PB-30

2.5 Logical Architecture

Inspired by the structure of the reference report, this section presents the logical architecture before the detailed sprint realization chapters. The project follows a **three-tier architecture**: a presentation tier for the React applications, a business tier for the Express.js API, and a data tier for PostgreSQL and Supabase services.

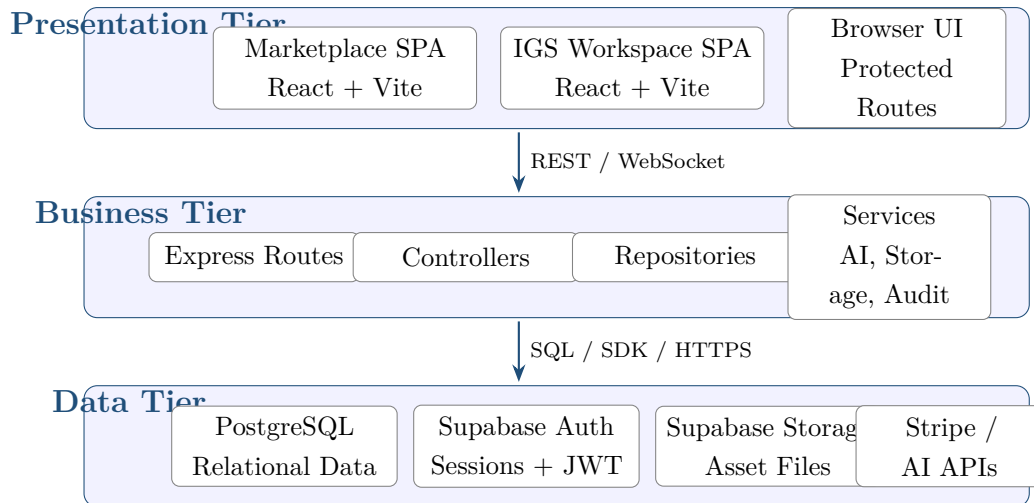


Figure 2.1: Three-Tier Logical Architecture of IGS Marketplace

2.5.1 Software Architecture

The backend is organized around a layered route-controller-repository structure. Routes expose REST endpoints and attach middleware. Controllers implement use cases and coordinate services. Repositories encapsulate SQL queries and protect the rest of the application from database details. Cross-cutting services such as storage, audit logging, recommendations, AI inference, and token management are reused by several controllers.

On the frontend, the Marketplace and IGS Workspace are independent React applications. They share the same backend API but keep their own routing, layout, page components, context providers, and services. This separation keeps the public marketplace experience distinct from the internal collaboration workspace.

2.6 Design Patterns

The codebase uses several recurring design patterns and architectural patterns:

Table 2.5: Design Patterns Used in the Project

Pattern	Project Location	Role in the System
Three-tier Architecture	React apps, Express API, PostgreSQL/Supabase	Separates presentation, business logic, and persistence concerns.
Layered Architecture	<code>routes</code> , <code>controllers</code> , <code>repositories</code>	Keeps HTTP routing, business rules, and SQL access independent.
Repository Pattern	<code>MarketPlace_Back/src/repositories</code>	Centralizes database operations for users, assets, orders, reviews, groups, trades, vouchers, and workspace data.
Middleware / Chain of Responsibility	<code>auth.js</code> , <code>permissions.js</code> , <code>validator.js</code> , <code>rateLimiters.js</code>	Applies authentication, authorization, validation, and rate limits before controllers execute.
Service Layer	<code>storageService</code> , <code>auditService</code> , <code>aiService</code> , <code>recommendationService</code>	Encapsulates reusable business services and external integrations.
Singleton Configuration	<code>database.js</code> , <code>supabase.js</code>	Shares one configured database pool and Supabase client across the back-end.
Provider Pattern	React context providers	Shares authentication, cart, permissions, notifications, filters, and user data across components.
Route Guard	<code>ProtectedRoute.jsx</code> , <code>Can.jsx</code>	Protects frontend pages and UI actions according to authentication and permissions.
Observer / Pub-Sub	Socket.io chat and notifications	Broadcasts chat messages and real-time events to subscribed clients.
Adapter Pattern	Frontend service files and backend external service wrappers	Hides API, payment, storage, and AI-provider details behind stable local functions.

2.7 Global Use Case Diagram

The global use case diagram must show the system boundary and the interactions between the main actors and the marketplace/workspace features. A draw.io-ready source is provided in `diagrams/drawio/use_case_global.mmd`.

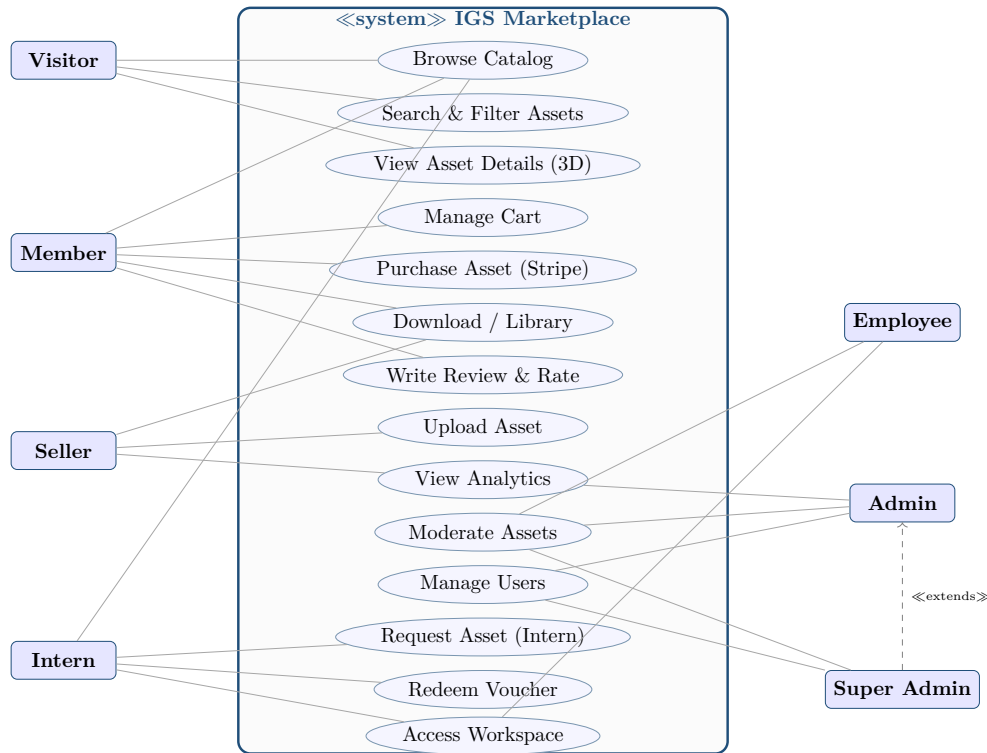


Figure 2.2: Global Use Case Diagram

2.8 Global Class Diagram

The class model is derived from the PostgreSQL schema and backend repositories. It focuses on the core domain objects: users, roles, assets, orders, reviews, moderation requests, vouchers, workspace projects, tasks, chat, groups, and trades. The draw.io-ready source is available in `diagrams/drawio/class_global.mmd`.

2.9 Key Sequence Diagrams

The most representative system flows are asset upload and moderation, purchase and library unlock, and intern asset request with voucher redemption. Their draw.io-ready sources are available under `diagrams/drawio/seq_*.mmd`. These diagrams should be generated in draw.io using Mermaid import, then exported as images or redrawn as native draw.io shapes if the final layout needs manual refinement.

Conclusion

This chapter clarified the scope of the platform, defined actors and requirements, and organized the work through a product backlog and sprint plan. It also prepared the UML artifacts needed for the rest of the report: global use case, class, and sequence views.

Sprint 0: Research, Planning & Architecture

Sommaire

3.1	System Architecture.....	22
3.2	Technology Stack Justification	23
3.3	Database Design	23

Introduction

Sprint 0 constitutes the foundational preparation phase before any code is written. It encompasses market research, requirements elicitation, architectural decision-making, technology selection, and database design. The deliverables of Sprint 0 are the design artefacts and infrastructure choices that guide all subsequent development sprints.

3.1 System Architecture

IGS Marketplace follows a **three-tier architecture** separating presentation, business logic, and data concerns. The two React + Vite SPAs (ports 5173 and 5174) communicate exclusively via a REST API with the Node.js/Express backend (port 3001), which in turn manages a PostgreSQL relational database and Supabase Storage for binary files.

The backend is internally structured around the **Routes → Controllers → Repositories** layered pattern with cross-cutting services (`storageService`, `auditService`, `aiService`, `tokenBlacklist`) used across multiple controllers.

3.2 Technology Stack Justification

Table 3.1: Sprint 0 — Technology Stack

Layer	Technology	Justification
Frontend	React 18 + Vite	Component model, rich ecosystem, fast HMR
3D Rendering	Three.js + Model Viewer	WebGL abstraction, native GLB/GLTF support
Backend	Node.js 20 + Express 4	Non-blocking I/O, unified JavaScript stack
Database	PostgreSQL 16	ACID compliance, JSONB columns, strong indexing
Authentication	Supabase Auth	Managed authentication, sessions, OAuth support, and JWT flow
File Storage	Supabase Storage	S3-compatible CDN, generous free tier
Payments	Stripe	International cards, PaymentIntent API
Real-time	Socket.io v4	Bidirectional events, room-based broadcasting
AI / LLM	HuggingFace Inference	Free-tier access to Mistral-7B open-source LLM

3.3 Database Design

The complete database schema was designed in Sprint 0 and implemented incrementally through 18 numbered migration scripts. The schema is organized into six conceptual groups:

1. **Identity & Access:** `users`, `user_roles`, `role_permissions`, `audit_logs`
2. **Catalog:** `assets`, `categories`, `asset_tags`, `asset_formats`
3. **Commerce:** `orders`, `order_items`, `cart_items`, `user_downloads`, `subscription_plans`, `user_subscriptions`
4. **Community:** `reviews`, `asset_likes`, `notifications`, `groups`, `group_members`, `trades`

5. **Workspace:** workspace_tasks, projects, chat_channels, chat_messages, direct_messages
6. **AI & Events:** user_events, intern_asset_requests, intern_vouchers

Conclusion

Sprint 0 established the architectural blueprint, technology choices, and database design on which the entire platform is built. The schema and project structure defined here were adhered to consistently throughout all subsequent sprints, minimizing rework and enabling rapid iterative delivery.

Sprints 1–2: Core Platform

Sommaire

4.1	Sprint 1: Authentication, Users and RBAC	26
4.1.1	Sprint Backlog	26
4.1.2	Analysis and Design	26
4.1.3	Implementation	27
4.1.4	Sprint Review	27
4.2	Sprint 2: Catalog and Marketplace Foundation	27
4.2.1	Sprint Backlog	27
4.2.2	Analysis and Design	28
4.2.3	Implementation	28
4.2.4	Sprint Review	28

Introduction

This chapter presents the first implementation phase of the project. Sprint 1 establishes the technical base of the platform: authentication, user management, RBAC, backend structure, database connection, and frontend route protection. Sprint 2 builds the public marketplace foundation: catalog navigation, asset details, category pages, upload preparation, cart, and payment flow.

4.1 Sprint 1: Authentication, Users and RBAC

4.1.1 Sprint Backlog

Sprint 1 – Core Access Management

Goal: implement a secure identity layer and role-based access control so that every future feature can rely on a stable permission model.

Table 4.1: Sprint 1 Backlog

ID	User Story	Priority
US-01	As a visitor, I can create an account and sign in securely.	High
US-02	As a user, I can access my profile and update my information.	High
US-03	As an administrator, I can assign roles and permissions.	High
US-04	As the system, I can protect API routes and frontend pages according to permissions.	High
US-05	As an administrator, I can trace sensitive access errors in audit logs.	Medium

4.1.2 Analysis and Design

The authentication flow relies on Supabase Auth sessions and JWT access tokens. After login, the frontend stores the authenticated state and sends the token to the backend through the `Authorization` header. The backend middleware verifies the token, loads the user record, attaches roles and permissions to `req.user`, and lets protected controllers continue only if the required permission is present.

The RBAC model is based on six roles: `member`, `seller`, `intern`, `employee`, `admin`, and `super_admin`. Permissions are stored separately and linked to roles through `role_permissions`, which makes the model extensible without changing route code.

4.1.3 Implementation

The backend implementation follows the layered structure introduced in Chapter 2: `auth.js` defines authentication endpoints, `authController.js` handles login, registration, logout, and role requests, while `userRepository.js` encapsulates SQL access for user data. Authorization rules are centralized in `permissions.js` and then reused by routes such as assets, orders, admin, workspace, chat, and intern requests.

On the frontend, `AuthContext.jsx`, `PermissionsContext.jsx`, `ProtectedRoute.jsx`, and `Can.jsx` provide route guards and conditional UI rendering. This avoids duplicating authorization checks in each page.

4.1.4 Sprint Review

At the end of Sprint 1, the platform had a reliable security foundation: users could authenticate, roles were assigned through the database, frontend pages could be protected, backend endpoints enforced permissions, and denied access attempts could be traced through the audit layer.

4.2 Sprint 2: Catalog and Marketplace Foundation

4.2.1 Sprint Backlog

Sprint 2 – Public Marketplace Foundation

Goal: deliver the first usable public marketplace experience: categories, listings, asset details, previews, cart, and checkout preparation.

Table 4.2: Sprint 2 Backlog

ID	User Story	Priority
US-06	As a visitor, I can browse all marketplace categories.	High
US-07	As a visitor, I can search and filter assets.	High
US-08	As a buyer, I can consult an asset detail page with previews and metadata.	High
US-09	As a member, I can add assets to a persistent cart.	High
US-10	As a member, I can start checkout and prepare payment.	High
US-11	As a seller, I can access the upload page according to permissions.	Medium

4.2.2 Analysis and Design

The catalog is organized around the `categories` and `assets` tables. Each asset belongs to a category and can have tags, metadata, formats, a thumbnail, a main file, and status information. The frontend category pages under `src/Pages/Types` reuse the same listing logic while applying different category filters.

Asset previews are adapted to the asset type. 3D models use `ModelViewer.jsx`, textures use `TextureViewer.jsx` and `TextureSphere.jsx`, HDRIs use `HdriViewer.jsx`, shaders use `ShaderPreview.jsx`, and scripts use `ScriptViewer.jsx`. This separates preview logic from the generic asset detail page.

4.2.3 Implementation

The backend exposes catalog features through `assets.js`, `assetController.js`, and `assetRepository.js`. Queries support listing, filtering, detail loading, download checks, likes, reviews, and AI summaries. Cart and order preparation are handled through `orders.js`, `orderController.js`, and `orderRepository.js`.

The frontend implements the public routes in `App.jsx`: `/Models`, `/Textures`, `/Shaders`, `/HDRIs`, `/Audio`, `/VFX`, `/Animations`, `/Environments`, `/Decals`, `/Cart`, `/Checkout`, and `/View/:id`. Shared services such as `assetService.js`, `cartService.js`, and `orderService.js` isolate API calls from components.

4.2.4 Sprint Review

Sprint 2 delivered the first complete user path: browse assets, open a detail page, preview the asset, add it to the cart, and start checkout. The work also prepared the later upload, moderation, and payment features by stabilizing the asset data model.

Conclusion

Sprints 1 and 2 delivered the technical and user-facing foundation of IGS Marketplace. The platform became secure, role-aware, navigable, and ready for the seller and administration workflows developed in the next phase.

Sprints 3–4: Marketplace Experience

Sommaire

5.1	Sprint 3: Upload, Moderation and Trust	30
5.1.1	Sprint Backlog	30
5.1.2	Analysis and Design	30
5.1.3	Implementation	31
5.1.4	Sprint Review	31
5.2	Sprint 4: Intern Workflow and Community Modules	31
5.2.1	Sprint Backlog	31
5.2.2	Analysis and Design	32
5.2.3	Implementation	32
5.2.4	Sprint Review	32

Introduction

This chapter presents the sprints that transform the catalog into a controlled marketplace. Sprint 3 focuses on seller upload, asset moderation, reviews, ratings, notifications, and subscriptions. Sprint 4 introduces role-specific business workflows: intern asset requests, vouchers, peer-to-peer trades, seller analytics, and community groups.

5.1 Sprint 3: Upload, Moderation and Trust

5.1.1 Sprint Backlog

Sprint 3 – Seller and Moderation Workflow

Goal: allow sellers to submit assets and allow internal reviewers to control catalog quality before publication.

Table 5.1: Sprint 3 Backlog

ID	User Story	Priority
US-12	As a seller, I can upload an asset with files, thumbnail, tags, metadata, and price.	High
US-13	As an admin, I can approve or reject submitted assets.	High
US-14	As a seller, I can request edits on approved assets.	Medium
US-15	As a buyer, I can rate and review purchased assets.	Medium
US-16	As a user, I can receive notifications about moderation and order events.	Medium
US-17	As a user, I can consult and subscribe to available plans.	Medium

5.1.2 Analysis and Design

The upload workflow is intentionally moderated. A submitted asset is created with a **pending** status and becomes visible in the public catalog only after approval. Rejected assets keep a rejection reason so the seller understands the correction needed. Approved assets cannot be edited freely; instead, sellers submit an edit request, which is reviewed by an authorized internal user.

Reviews and ratings are linked to ownership. This prevents users from reviewing assets that they have not purchased or unlocked. Ratings update aggregated counters on the asset record, while reviews can also receive AI sentiment metadata.

5.1.3 Implementation

File upload and storage are handled by the upload route and the storage service. Supabase Storage keeps binary asset files and thumbnails, while PostgreSQL stores meta-data, moderation status, tags, formats, and ownership information. Admin moderation is implemented under `admin.js` and `adminController.js`; asset edit requests use `assetEditRequestRepository.js`.

The review system uses `reviewController.js` and `reviewRepository.js`. The notification system stores persistent notifications in PostgreSQL and exposes notification routes used by the frontend `NotificationContext.jsx` and navigation panel.

5.1.4 Sprint Review

At the end of Sprint 3, the marketplace had a credible trust layer: assets could be submitted and moderated, users could review owned assets, administrators had moderation tools, and notifications kept users informed about workflow changes.

5.2 Sprint 4: Intern Workflow and Community Modules

5.2.1 Sprint Backlog

Sprint 4 – Role-Specific Advanced Features

Goal: make the RBAC model operational through intern requests, vouchers, groups, trades, and seller-oriented analytics.

Table 5.2: Sprint 4 Backlog

ID	User Story	Priority
US-18	As an intern, I can request access to a paid asset with a justification.	High
US-19	As an admin or employee, I can approve or reject intern asset requests.	High
US-20	As an admin, I can generate vouchers for controlled asset access.	Medium
US-21	As a user, I can redeem a valid voucher.	Medium
US-22	As a user, I can create or join groups.	Low
US-23	As a user, I can propose an asset trade to another user.	Low

5.2.2 Analysis and Design

The intern workflow was designed for a studio context where interns may need paid assets but should not purchase them personally. The request stores the intern, target asset, message, status, reviewer, and review note. After approval, a voucher can be created and redeemed to unlock the asset in the user’s library.

Groups and trades add a community dimension to the marketplace. Groups organize users around shared interests, while trades allow users to propose exchanges between assets.

5.2.3 Implementation

Intern requests are implemented through `intern.js`, `internAssetRequestRepository.js`, and admin moderation endpoints. Voucher logic is encapsulated in `voucherRepository.js`. Groups and trades are implemented through `groups.js`, `trades.js`, `groupController.js`, and `tradeController.js`.

Frontend pages such as `VoucherRedeem.jsx`, `Group.jsx`, `Trade.jsx`, and `Analytics.jsx` expose these workflows while respecting route guards and permissions.

5.2.4 Sprint Review

Sprint 4 completed the advanced marketplace workflows. The platform became more than a catalog: it supported internal asset access, controlled vouchers, community interaction, and seller-oriented visibility.

Conclusion

Sprints 3 and 4 introduced the governance and collaboration mechanisms required for a professional asset marketplace. They strengthened trust, moderation, internal control, and community engagement.

Sprints 5–6: Workspace and AI

Sommaire

6.1	Sprint 5: IGS Workspace	34
6.1.1	Sprint Backlog	34
6.1.2	Analysis and Design	34
6.1.3	Implementation	35
6.1.4	Sprint Review	35
6.2	Sprint 6: AI Services	35
6.2.1	Sprint Backlog	35
6.2.2	Analysis and Design	35
6.2.3	Implementation	36
6.2.4	Sprint Review	36

Introduction

This chapter presents the two sprints that extend IGS Marketplace into a broader studio ecosystem. Sprint 5 introduces the IGS Workspace, a private internal application for team, project, task, and chat management. Sprint 6 integrates AI services for recommendations, review analysis, and chatbot assistance.

6.1 Sprint 5: IGS Workspace

6.1.1 Sprint Backlog

Sprint 5 – Internal Collaboration Workspace

Goal: provide IGS employees and interns with an internal workspace connected to the same authentication and permission system as the marketplace.

Table 6.1: Sprint 5 Backlog

ID	User Story	Priority
US-24	As an internal user, I can access a dedicated workspace dash-board.	High
US-25	As a manager, I can manage team members and positions.	High
US-26	As an internal user, I can create and follow tasks.	High
US-27	As an internal user, I can create projects and assign members.	High
US-28	As a project member, I can attach project documents and view progress.	Medium
US-29	As an internal user, I can chat through channels and direct messages.	Medium

6.1.2 Analysis and Design

The IGS Workspace is implemented as a separate React + Vite application under `IGS_Workspace`. It has its own layout, sidebar, top bar, pages, services, and contexts, but consumes the same backend and identity layer as the marketplace. This design preserves a clear separation between public marketplace workflows and internal studio workflows.

Workspace data is stored in dedicated tables: `projects`, `project_members`, `project_documents`, `workspace_tasks`, `chat_channels`, `chat_messages`, and related chat tables.

6.1.3 Implementation

The backend workspace routes are grouped in `workspace.js`. They expose endpoints for team management, dashboard statistics, tasks, projects, project members, project documents, and project-specific tasks. Real-time chat is implemented through `chat.js`, `chatController.js`, `chatRepository.js`, and `Socket.io`.

The frontend workspace includes pages such as Dashboard, Projects, Project Detail, Tasks, Team, Chat, Assets, Users, Roles, Orders, Calendar, Reports, Performance, and Settings. Shared services such as `workspaceService.js` and `api.js` isolate backend communication.

6.1.4 Sprint Review

Sprint 5 delivered the internal application needed by the studio. Internal users could manage projects, tasks, teams, and chat in a single environment while keeping access restricted to authorized roles.

6.2 Sprint 6: AI Services

6.2.1 Sprint Backlog

Sprint 6 – AI Assistance

Goal: enrich the platform with AI-supported analysis, recommendations, and conversational assistance.

Table 6.2: Sprint 6 Backlog

ID	User Story	Priority
US-30	As an admin, I can view AI sentiment information for reviews.	Medium
US-31	As a user, I can receive personalized recommendations.	Medium
US-32	As a visitor or member, I can ask marketplace questions through a chatbot.	Medium
US-33	As the system, I can record user events for recommendation input.	Medium

6.2.2 Analysis and Design

The AI layer is integrated as a service layer rather than being embedded directly into controllers. This keeps the platform flexible: the provider or model can evolve

without rewriting marketplace routes. User events such as views, likes, downloads, and purchases are stored in `user_events` and used by the recommendation service.

6.2.3 Implementation

AI logic is grouped in `aiService.js`, `intentService.js`, and `recommendationService.js`. The chatbot is exposed through `bot.js` and `botController.js`. Review analysis is connected to the review workflow, while recommendations are generated from stored user interactions and asset metadata.

On the frontend, `BotWidget.jsx` provides a floating assistant interface and marketplace pages can consume recommendation results without knowing the implementation details of the AI provider.

6.2.4 Sprint Review

Sprint 6 added intelligent assistance to the marketplace. The system could support review moderation, recommend relevant assets, and answer user questions through a conversational interface.

Conclusion

Sprints 5 and 6 expanded IGS Marketplace beyond commercial workflows. The platform gained an internal studio workspace and AI-assisted features that improve collaboration, discovery, and moderation.

Sprints 7–8: IGS Workspace & Admin Dashboard

Sommaire

7.1	Sprint 8: Admin Dashboard & Premium UI Overhaul.....	38
7.1.1	Sprint Backlog.....	38
7.1.2	Implementation.....	39
7.1.3	Sprint Review — Deliverables.....	39

Introduction

Chapter 7 covers Sprints 7 and 8. Sprint 7 completes the AI layer with a personalized recommendation engine and a floating marketplace chatbot, expands the asset catalog to 13 industry-standard categories, and populates every category with real seed data hosted on Supabase CDN. Sprint 8 consolidates the admin dashboard, delivers the premium UI overhaul, and completes all placeholder workspace pages before the production release.

7.1 Sprint 8: Admin Dashboard & Premium UI Overhaul

Introduction

Sprint 8 completes the platform with a polished administration interface, a redesigned premium marketplace UI, and final quality-of-life improvements identified during the Sprint 7 review.

7.1.1 Sprint Backlog

Sprint 8 — Final Sprint

Goal: Deliver a fully polished admin dashboard, premium visual design, and all remaining platform refinements for the final defense demonstration.

Table 7.1: Sprint 8 — User Stories

ID	User Story	Priority	Status
US-53	As an admin, I want a consolidated dashboard with key platform KPIs at a glance.	High	Done
US-54	As a user, I want the marketplace to have a premium, professional visual identity.	High	Done
US-55	As a user, I want a real-time notification panel accessible from any page.	Medium	Done
US-56	As any user, I want all placeholder Workspace pages to display meaningful content.	Medium	Done

7.1.2 Implementation

Premium UI Overhaul

The marketplace frontend received a complete visual redesign: refined typography, a cohesive color system using the IGS brand palette, polished card components with hover animations, and a redesigned navigation bar with a sliding real-time notification panel.

Admin Dashboard Consolidation

The admin dashboard was extended with KPI summary cards displaying: total registered users, total assets by status, total orders and revenue, and active seller count. A recent-activity feed lists the last 20 platform events in reverse chronological order.

Workspace Completion

All placeholder pages in the IGS Workspace (Team, Calendar, Reports, Performance, Orders, Roles & Permissions, System Settings) were replaced with functional content layouts, completing the internal dashboard for the demonstration.

7.1.3 Sprint Review — Deliverables

- Premium visual overhaul of the Marketplace frontend across all major pages.
- Consolidated admin dashboard with KPI cards and activity feed.
- Real-time notification panel integrated into the navigation bar.
- All IGS Workspace placeholder pages replaced with complete content layouts.
- Final bug fixes and responsive design adjustments across both applications.

Conclusion

Sprint 8 brought the platform to production-ready quality. The premium UI, consolidated admin tools, and complete workspace navigation ensure that IGS Marketplace presents a professional, fully operational system for the jury defense.

CI/CD & Deployment

Sommaire

8.1	CI/CD Pipeline	41
8.1.1	Deployment Pipeline	41
8.2	Containerization with Docker	41
8.3	Nginx Reverse Proxy	42
8.4	Deployment Strategy	42
8.4.1	Image Publication and Remote Update	42
8.4.2	Database Migrations	42
8.4.3	Environment Secrets	43
8.5	API Documentation	43
8.6	Testing Strategy	44

Introduction

This chapter describes the Continuous Integration and Continuous Delivery pipeline implemented for IGS Marketplace, the Docker-based containerization strategy, and the production deployment architecture.

8.1 CI/CD Pipeline

The pipeline is implemented with **GitHub Actions**, triggered on every push or pull request targeting `main`. It runs three sequential jobs:

1. **lint-build** — static analysis and compilation check for frontend and backend.
2. **test** — automated test suite execution (needs: lint-build).
3. **docker-publish & deploy** — builds Docker images, pushes to DockerHub, and updates the production server via SSH (only on `main` branch pushes).

8.1.1 Deployment Pipeline

The diagram below illustrates the full flow from a developer commit to a live production update:



Figure 8.1: IGS Marketplace Deployment Pipeline

The GitHub Actions runner authenticates to the production server using an RSA key pair (4096-bit) stored as a GitHub Actions Secret. After Docker images are pushed to DockerHub, the runner connects via SSH and instructs `docker-compose` to pull the latest images and restart the containers with zero manual intervention.

8.2 Containerization with Docker

Docker packages each service into a reproducible image, guaranteeing identical behavior across development, CI, and production environments.

- **Backend (Express.js):** A Node 20 Alpine image installs production dependencies only, copies the source, and starts the server on port 3001.

- **Frontend (React/Vite):** A multi-stage build — the first stage compiles the Vite bundle inside a Node container; the second stage copies the `dist/` output into a lightweight Nginx image for static serving.
- **Database:** The official PostgreSQL 16 Alpine image with a persistent named volume for data durability.

A single `docker-compose.yml` orchestrates all three services (`db`, `backend`, `frontend`) on an isolated Docker network, with `depends_on` ensuring startup order. Environment variables are injected from a server-side `.env` file at runtime — no credentials are committed to the repository.

8.3 Nginx Reverse Proxy

Nginx serves as both the static file server for the React bundle and a reverse proxy routing `/api/*` requests to the Express backend container. This single entry point simplifies TLS termination and eliminates browser CORS issues. The `try_files` directive ensures React Router's client-side routes are handled by the SPA rather than returning a 404 from Nginx.

8.4 Deployment Strategy

8.4.1 Image Publication and Remote Update

On every successful pipeline run on `main`, the workflow authenticates to DockerHub using stored secrets and pushes fresh `igs-backend` and `igs-frontend` images tagged `latest`. The deploy job then SSHs into the production server and runs `docker-compose pull && docker-compose up -d --remove-orphans`, delivering zero-downtime rolling updates.

8.4.2 Database Migrations

Schema migrations are applied sequentially using numbered SQL files (`001_*.sql` through `018_*.sql`). When a deployment introduces schema changes, the migration is executed inside the `db` container before the backend restarts, keeping schema and application code in sync.

8.4.3 Environment Secrets

Table 8.1: GitHub Actions Secrets — Classification

Secret	Service	Purpose
SUPABASE_JWT_SECRET	Supabase Auth	JWT verification and protected API access
STRIPE_SECRET_KEY	Stripe	Payment processing & webhooks
SUPABASE_URL / KEY	Supabase Storage	Asset file CDN upload/download
JWT_SECRET	Express middleware	Session signing
HUGGINGFACE_API_TOKEN	HuggingFace Hub	AI chatbot inference calls
DB_PASSWORD	PostgreSQL	Database authentication
SSH_PRIVATE_KEY	GitHub Actions	Secure server access

8.5 API Documentation

The REST API is documented using **Postman Collections**, organized by resource domain. Each collection includes pre-configured authentication headers, example request bodies, expected response schemas, and environment variables to switch between local and production targets.

Table 8.2: API Documentation — Collections by Domain

Collection	Base path	Endpoints covered
Authentication	/api/auth	Register, login, refresh, logout
Assets	/api/assets	CRUD, upload, search, filter
Orders	/api/orders	Create, status, history
Cart	/api/cart	Add, remove, clear, checkout
Users	/api/users	Profile, roles, admin management
Payments	/api/payments	Stripe intent, webhook handler
Reviews	/api/reviews	Submit, list, AI sentiment score
Notifications	/api/notifications	List, mark-read, real-time (WS)

8.6 Testing Strategy

Table 8.3: Test Coverage by Layer

Layer	Tool	Coverage
Backend routes	Postman collections	All CRUD endpoints per resource
Authentication	JWT and RBAC middleware	Token expiry, role escalation
Payment integration	Stripe test mode	Success, failure, and webhook scenarios
Frontend	Manual browser testing	Golden path + edge cases per sprint
AI models	Python unit tests	Intent classifier, sentiment analyser
Containers	docker-compose smoke test	All services start, health checks pass

Conclusion

The GitHub Actions CI/CD pipeline provides end-to-end automation from commit to production. Docker containerization ensures environment consistency, Nginx consolidates the frontend and API behind a single entry point, and GitHub Secrets ensure no credential ever reaches the repository. This infrastructure gives the platform a production-grade, reproducible, and maintainable deployment foundation.

General Conclusion

IGS Marketplace was successfully designed and developed over eight sprints, delivering Tunisia's first dedicated 3D digital asset platform. The system provides a six-level Role-Based Access Control system, an interactive in-browser 3D asset viewer, Stripe payment integration, AI-powered review sentiment analysis, a personalized recommendation engine, a floating chatbot assistant powered by Mistral-7B-Instruct, and a companion internal workspace application — all within a cohesive full-stack ecosystem built on React, Node.js/Express, and PostgreSQL.

Several key technical insights emerged throughout the project. Designing the permission system during Sprint 0 and enforcing it consistently across all subsequent sprints was the single most impactful architectural decision — retrofitting access control would have been significantly more costly. The three-tier layered architecture (Routes → Controllers → Repositories) proved highly maintainable for a REST API at this level of complexity, and applying Scrum as a solo developer, with supervisors occupying the Product Owner and Scrum Master roles, effectively structured six months of work into reviewable, deliverable-focused increments.

Looking ahead, several directions are planned for the platform's continued evolution: a React Native mobile client, a full automated test suite using Jest and Supertest, expanded AI capabilities including streaming chatbot responses and AI-based tag suggestions, a digital asset licensing framework where sellers can define usage rights, and production deployment on a cloud provider with the Docker-based CI/CD pipeline fully active.

Bibliography

- [1] Roques, P. *UML 2 par la Pratique — Études de cas et exercices corrigés*. 8^e édition. Eyrolles, 2014.
- [2] Schwaber, K. & Sutherland, J. *The Scrum Guide*. November 2020. <https://scrumguides.org/scrum-guide.html>
- [3] Object Management Group. *UML 2.5.1 Specification*. 2017. <https://www.omg.org/spec/UML/2.5.1>
- [4] Jones, M. et al. *RFC 7519 — JSON Web Token (JWT)*. IETF, 2015. <https://datatracker.ietf.org/doc/html/rfc7519>
- [5] Ferraiolo, D. & Kuhn, R. *Role-Based Access Control*. NIST, 2009.
- [6] P Market Research. *Worldwide 3D Marketplace Models Market Research 2024 — by Type, Application, Participants and Countries, Forecast to 2030*. 2024. <https://pmarketresearch.com/worldwide-3d-marketplace-models-market-research-2024-by-type-application-parti>
- [7] Citius Research. *3D Project Market Report*. 2024. <https://citiusresearch.com/publication-details/3d-project-market-report>

Webography

1. OpenJS Foundation. *Node.js Official Documentation*.
<https://nodejs.org/en/docs/> [Accessed April 2026]
2. StrongLoop / OpenJS Foundation. *Express.js Application Guide*.
<https://expressjs.com/en/guide/routing.html> [Accessed April 2026]
3. Meta Platforms. *React 18 — Official Documentation*.
<https://react.dev/> [Accessed April 2026]
4. The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*.
<https://www.postgresql.org/docs/16/> [Accessed April 2026]
5. Supabase Inc. *Supabase Documentation — Storage*.
<https://supabase.com/docs/guides/storage> [Accessed April 2026]
6. Stripe, Inc. *Stripe API Reference — Payment Intents*.
https://stripe.com/docs/api/payment_intents [Accessed April 2026]
7. Supabase Inc. *Supabase Documentation — Auth*.
<https://supabase.com/docs/guides/auth> [Accessed April 2026]
8. Socket.IO contributors. *Socket.IO v4 Documentation*.
<https://socket.io/docs/v4/> [Accessed April 2026]
9. Google LLC. *Model Viewer — Web Component for 3D Models*.
<https://modelviewer.dev/> [Accessed April 2026]
10. Anthropic. *Claude API Documentation — Messages & Tool Use*.
<https://docs.anthropic.com/> [Accessed April 2026]
11. Vitejs. *Vite — Next Generation Frontend Tooling*.
<https://vitejs.dev/guide/> [Accessed April 2026]
12. Flouci. *Flouci Developer Documentation — Payment Gateway (Tunisia)*.
<https://developer.flouci.com/> [Accessed April 2026]
13. OWASP Foundation. *OWASP Top Ten — Web Application Security Risks*.
<https://owasp.org/www-project-top-ten/> [Accessed April 2026]

Abstract

ملخص

يقدم هذا المشروع تصميم وتطوير منصة **IGS Marketplace** لفائدة شركة **Studio Games Inherited**، وهي شركة تونسية متخصصة في تطوير ألعاب الفيديو. تهدف المنصة إلى توفير فضاء موحد لنشر وبيع وشراء الأصول الرقمية الخاصة بالألعاب، مثل النماذج ثلاثية الأبعاد، الخامات، المؤثرات، الإضافات، الملفات الصوتية، السكريبتات وملفات الطباعة ثلاثية الأبعاد. كما توفر المنصة نظام صلاحيات مبنيا على الأدوار، مسار مصادقة آمن، عارضا تفاعليا للأصول، إدارة للسلة والطلبات، دفعا إلكترونيا، مكتبة للمشتريات، تقييما ومراجعات، إشعارات، اشتراكات، قسائم، ومجمعا للتبادل بين المستخدمين. ويتضمن المشروع كذلك فضاء داخليا باسم **IGS Workspace** لإدارة المشاريع والمهام والفريق والمحادثات الفورية. تم الاعتماد على **React** و **Vite** في الواجهة، و **Node.js** و **Express.js** في الخادم، مع **PostgreSQL** و **Supabase** للتخزين، إضافة إلى خدمات ذكاء اصطناعي للمراجعات والتوصيات والمساعد الحواري. أنجز العمل وفق منهجية **Scrum** عبر ثمانية سبرنتات.

الكلمات المفتاحية: منصة أصول رقمية، ألعاب فيديو، **React**، **Node.js**، **PostgreSQL**، **RBAC**، **Scrum**، ذكاء اصطناعي.

English

This end-of-studies project presents the design and development of **IGS Marketplace**, a full-stack digital asset marketplace built for **Inherited Games Studio**, a Tunisian video game development company. The platform centralizes the publication, discovery, purchase, and management of game-ready digital assets such as 3D models, textures, HDRIs, shaders, plugins, audio files, scripts, visual effects, animations, and 3D-printable files. It includes secure authentication, role-based access control, catalog search and filtering, interactive asset previews, seller upload and moderation workflows, cart and order management, online payment, library downloads, reviews and ratings, notifications, subscriptions, vouchers, groups, and peer-to-peer trades. The project also integrates **IGS Workspace**, an internal collaboration application for team management, projects, tasks, documents, dashboards, and real-time chat. The solution was implemented with **React**, **Vite**, **Node.js**, **Express.js**, **PostgreSQL**, **Supabase**, and AI services for review analysis, recommendations, and chatbot assistance. The project was managed with the **Scrum** methodology over eight sprints.

Keywords: Digital Asset Marketplace, Video Games, React, Node.js, PostgreSQL, RBAC, Scrum, Artificial Intelligence.

Résumé

Ce projet de fin d'études présente la conception et le développement de **IGS Marketplace**, une plateforme web full-stack réalisée pour **Inherited Games Studio**, une société tunisienne spécialisée dans le développement de jeux vidéo. La solution centralise la publication, la recherche, l'achat et la gestion d'assets numériques destinés aux jeux, tels que les modèles 3D, textures, HDRIs, shaders, plugins, fichiers audio, scripts, effets visuels, animations et fichiers imprimables.

en 3D. La plateforme offre l'authentification sécurisée, le contrôle d'accès par rôles, la recherche et le filtrage du catalogue, les aperçus interactifs, le dépôt d'assets par les vendeurs, la modération, le panier, les commandes, le paiement en ligne, la bibliothèque de téléchargements, les avis, les notes, les notifications, les abonnements, les bons d'accès, les groupes et les échanges entre utilisateurs. Le projet inclut également **IGS Workspace**, une application interne dédiée à la gestion de l'équipe, des projets, des tâches, des documents, des tableaux de bord et du chat en temps réel. La solution repose sur **React**, **Vite**, **Node.js**, **Express.js**, **PostgreSQL**, **Supabase** et des services d'intelligence artificielle pour l'analyse des avis, les recommandations et le chatbot. Le travail a été conduit selon la méthodologie **Scrum** sur huit sprints.

Mots-clés : Marketplace d'assets numériques, jeux vidéo, React, Node.js, PostgreSQL, RBAC, Scrum, intelligence artificielle.
